

数据处理优化:

clean_complex_features_optimized,

clean_complex_features_rent

在开头就使用与定义两组数据的函数，处理各种格式，防止后续流程price和rent反复使用造成的臃肿；

根据源数据对数据预处理进行了补充，补充了环线、朝向、电梯、付款等特征。

特征工程与预处理:

KNN优化:挖掘“邻居价格”来突破树模型对经纬度处理的局限。

TF-IDF + SVD: 先把‘房屋优势’、‘核心卖点’等列进行文本融合，对融合后的文本分别先后使用TF-IDF + SVD的方法捕捉具体特征。

ANN:构建一个稳健的深度学习模型，随层数加深神经元数量逐渐减少，为防止梯度爆炸、过拟合等问题使用归一化，为了保证模型稳定把学习率降低到0.0005.损失函数使用的是MAE，与本作业要优化的目标保持一致。

```
def build_ann_model(input_dim):
    inputs = layers.Input(shape=(input_dim,))

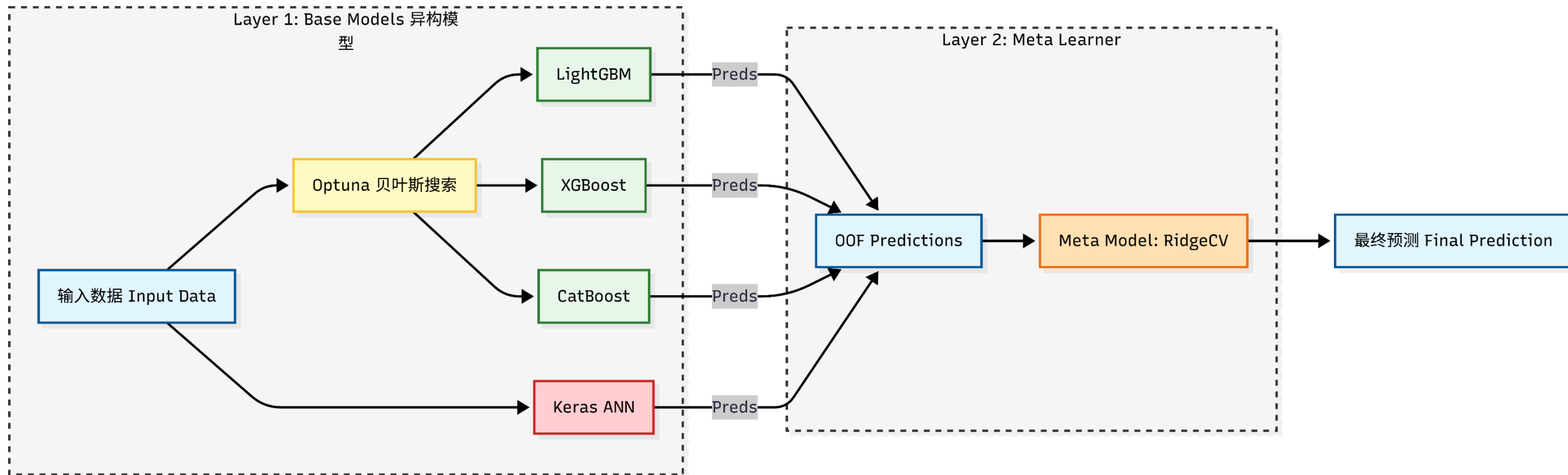
    x = layers.Dense(1024, activation='relu')(inputs)
    x = layers.BatchNormalization()(x)
    x = layers.Dropout(0.3)(x)
    x = layers.Dense(512, activation='relu')(x)
    x = layers.BatchNormalization()(x)
    x = layers.Dropout(0.3)(x)
    x = layers.Dense(256, activation='relu')(x)
    x = layers.BatchNormalization()(x)
    x = layers.Dropout(0.2)(x)
    x = layers.Dense(128, activation='relu')(x)
    x = layers.BatchNormalization()(x)
    x = layers.Dropout(0.1)(x)
    x = layers.Dense(64, activation='relu')(x)
    x = layers.BatchNormalization()(x)

    outputs = layers.Dense(1)(x)
    model = keras.Model(inputs=inputs, outputs=outputs)
    optimizer = keras.optimizers.Adam(learning_rate=0.0005)
    model.compile(optimizer=optimizer, loss='mean_absolute_error')
    return model
```

Optuna: 为了确保基模型足够强大，引入 Optuna 进行贝叶斯超参数优化。这替代了传统的手动调参，自动找到了 LightGBM、XGBoost 和 CatBoost 的全局最优参数。

异构架构: 不只使用了树模型，还集成了前面的5层的深度神经网络 (ANN)。树模型擅长处理结构化数据，而神经网络能捕捉树模型忽略的非线性特征，两者互补性极强。

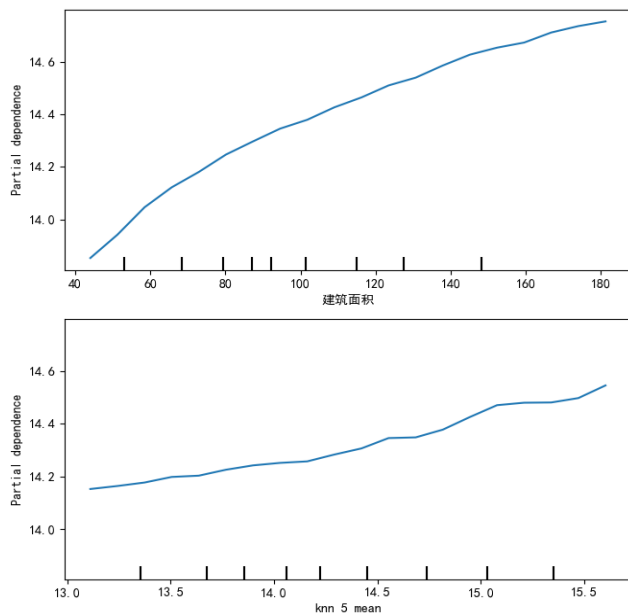
Ridge: 作为元学习器融合预测结果，通过6折交叉验证生成的 OOF 数据进行训练，在数学上保证了模型的稳健性，避免过拟合。



03 结果展示

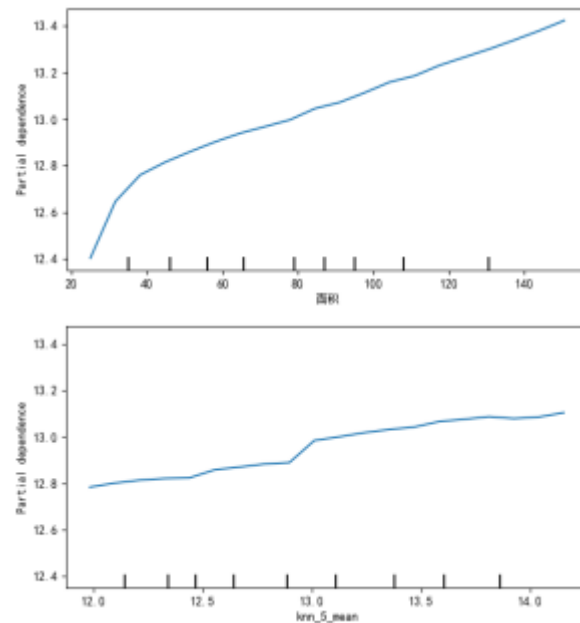
pdp_plot_price:

随着建筑面积、knn_5_mean（周围邻居均价）等特征，这些图表是上升趋势。特征数值变大，房价也随之变高。



pdp_plot_rent:

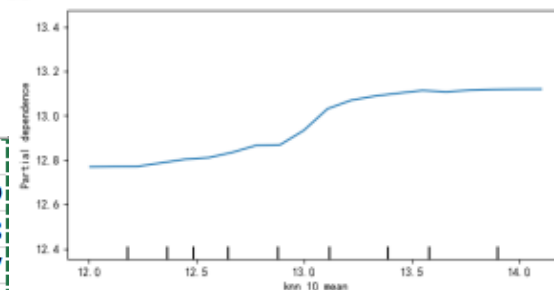
随着面积、knn_5_mean、knn_10_mean（周围邻居均价）等特征上升，这些图表是上升趋势。特征数值变大，房价也随之变高。



以Price为例：最佳是集成模型。交叉验证 MAE 是衡量模型泛化能力（在未知数据上表现）最重要的指标，因此选择集成是合理的；过拟合处理较好（和CV相似）

Price

Metrics	In-sample MAE	In-sample RMSE	Out-of-sample MAE	Out-of-sample RMSE	Cross-validation MAE	Cross-validation RMSE
Midterm(Ridge)	435525.7591	1021151.367	438027.6751	1033488.151	487918.1707	1103187.589
LightGBM	109543.4888	351500.398	221729.3315	489200.1756	220305.5866	508674.9688
XGBoost	134493.1743	351580.7742	225602.4462	485676.5287	221667.4477	496785.8627
CatBoost	141877.9084	371522.6804	222198.7339	479391.3176	219923.0687	492183.3742
Random Forest	226195.0743	455430.1751	303476.2489	684300.5793	302731.7988	702048.8396
Stacking Model	124064.4969	339210.8035	214881.885	467681.2858	211791.9209	476243.9308
ANN	232408.6072	553631.3035	269070.2166	614618.1031	351489.6072	802472.6373
Best Model	124064.4969	339210.8035	214881.885	467681.2858	211791.9209	476243.9308



Rent

Metrics	In-sample MAE	In-sample RMSE	Out-of-sample MAE	Out-of-sample RMSE	Cross-validation MAE	Cross-validation RMSE
Midterm(OLS)	100105.9787	212061.0567	99040.76727	211101.8987	100311.8726	212368.4196
LightGBM	39610.9815	111144.5687	67397.40757	153943.6111	67241.31352	150711.0976
XGBoost	27358.51536	86393.29036	67580.32179	157464.2873	67631.92663	153454.7806
CatBoost	53506.53204	130993.9052	67932.44713	154887.3985	68131.62756	153869.5481
Random Forest	76572.87577	159094.8157	87368.40126	201854.2119	87131.5908	198860.4074
Stacking Model	37118.05198	98080.6079	66100.81879	150877.3185	66052.62829	147587.4946
ANN	83336.25618	193455.6405	84419.35142	193186.1721	91652.74854	212059.0983
Best Model	37118.05198	98080.6079	66100.81879	150877.3185	66052.62829	147587.4946